Regularization cho mạng neural

Hạn chế học nhiễu, quản lý trạng thái layer

Chương 3 · Bài 3.6

Train tốt, bài mới kém

16 điểm có nhiễu, mạng hai lớp ẩn.

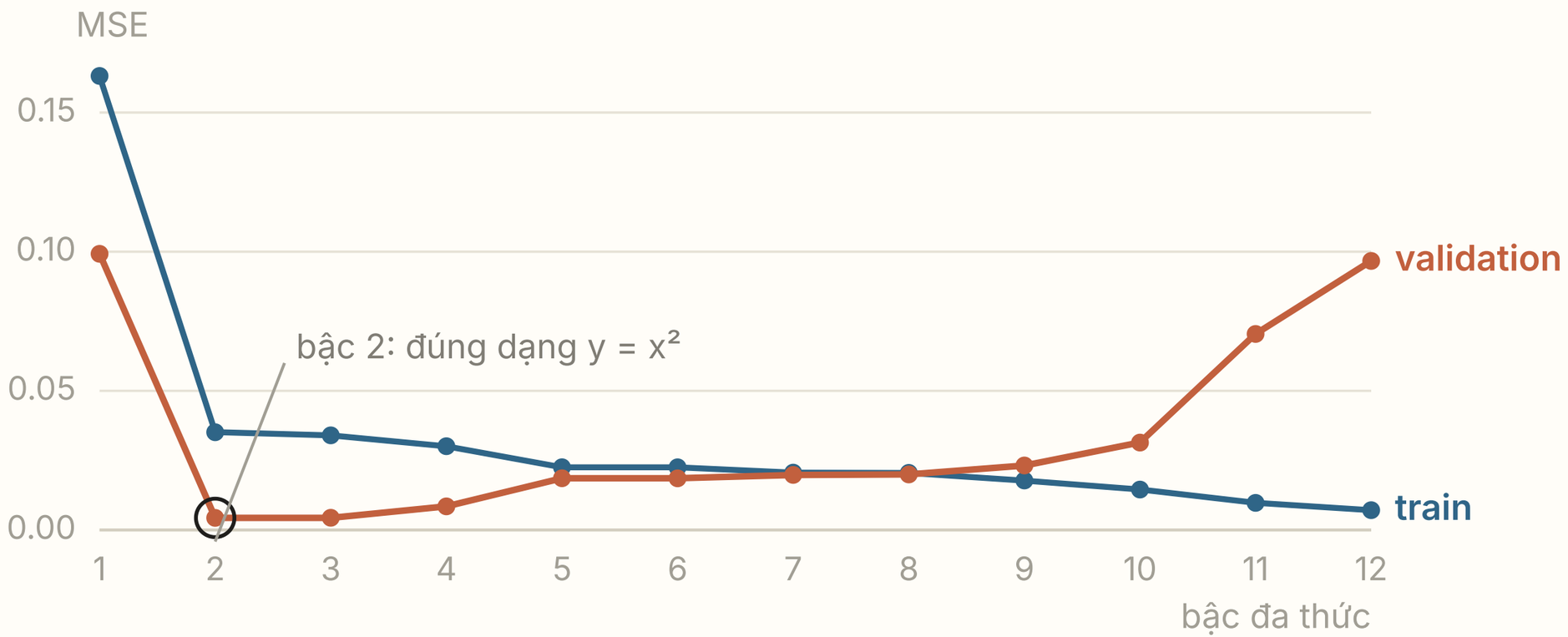
Epoch 26: validation tốt nhất. Epoch 160: tệ hơn ba lần.

Dropout → activation · decay → tham số.

Early stopping → thời điểm dừng.

BatchNorm: trạng thái riêng theo train/eval.

Khả năng biểu diễn tăng



16 điểm có nhiễu quanh y = x²: tăng bậc đa thức thì train MSE giảm mãi, còn validation MSE thấp nhất ở bậc 2–3 rồi tăng vọt khi model bám cả nhiễu.

Capacity ở đây là bậc đa thức; với mạng neural, cùng hiện tượng xảy ra theo epoch.

HỎI NHANH

Một đội chia ngẫu nhiên các khung hình lấy từ cùng một bộ video, nên validation có nhiều ảnh gần trùng với ảnh train. Train loss là 0.02, validation loss là 0.03. Kết luận nào hợp lý?

A Model tổng quát hóa tốt, có thể bỏ regularization

B Nên thêm regularization, vì khoảng cách vẫn còn dương

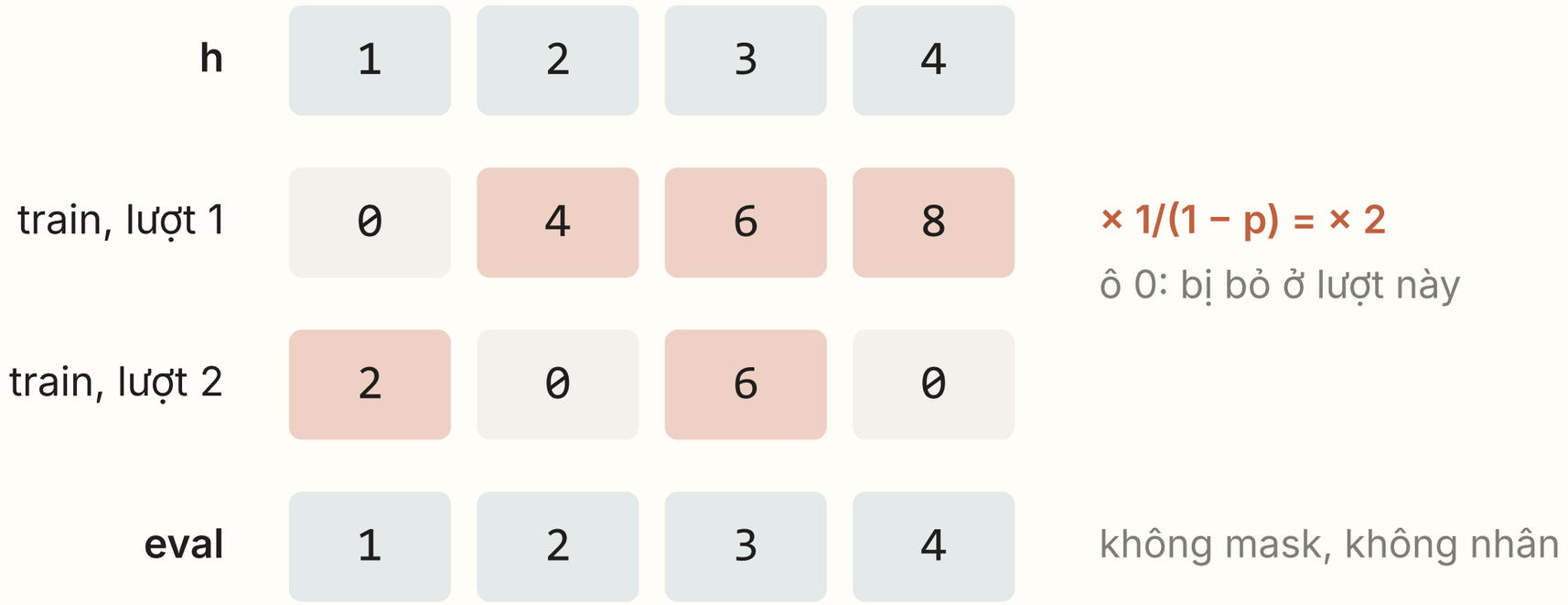
C Chưa kết luận được gì, phải chia lại theo video trước

D Validation thấp như vậy nghĩa là model đang underfit

01

Dropout và decay

Bỏ một số đường trong train



Dropout p = 0.5 trên h = (1, 2, 3, 4), output của code bài: mỗi lượt train bốc một mask mới và nhân phần còn lại với 2; eval giữ nguyên h.

Kỳ vọng không đổi

, .

Đầu ra 0 hoặc 6, mỗi giá trị xác suất 1/2.

Hệ số bù ở train nên eval giữ nguyên.

Từng mask không giữ đúng một nửa số phần tử.

Kết quả hai lượt

KẾT QUẢ

```
train 1: [0.0, 4.0, 6.0, 8.0]
train 2: [2.0, 0.0, 6.0, 0.0]
eval: [1.0, 2.0, 3.0, 4.0]
```

Train bốc mask mới.

Eval không bốc mask và không co thêm.

`no_grad` không tự tắt dropout.

HỎI NHANH

Dropout với . Một activation bằng 4 được giữ lại trong lượt train. Sau dropout, giá trị của nó là bao nhiêu?

A 3.2

B 4

C 5

D 20

HỎI NHANH

Một đội tự cài dropout theo kiểu bài báo gốc: lúc train chỉ nhân mask với , không chia cho . Khi eval tắt mask, phải làm gì để thang activation khớp với lúc train?

A Giữ nguyên activation, như inverted dropout

B Nhân activation với

C Chia activation cho

D Vẫn bốc mask như lúc train

Decay tác động chỗ khác

Dropout: activation ngẫu nhiên.

Decay: co tham số mỗi step

Rate đổi theo scheduler thì hệ số co đổi theo.

Quá mạnh: cả train lẫn validation kém.

Dò trên thang log, chọn bằng validation.

HỎI NHANH

AdamW với `weight_decay=0.1`. Scheduler giảm learning rate từ 0.01 xuống 0.001 ở epoch 30. Hệ số co trọng số ở mỗi bước thay đổi thế nào?

A Giữ ở , vì `weight_decay` không đổi

B Từ thành

C Từ thành

D Từ thành

02

Chọn thời điểm dừng

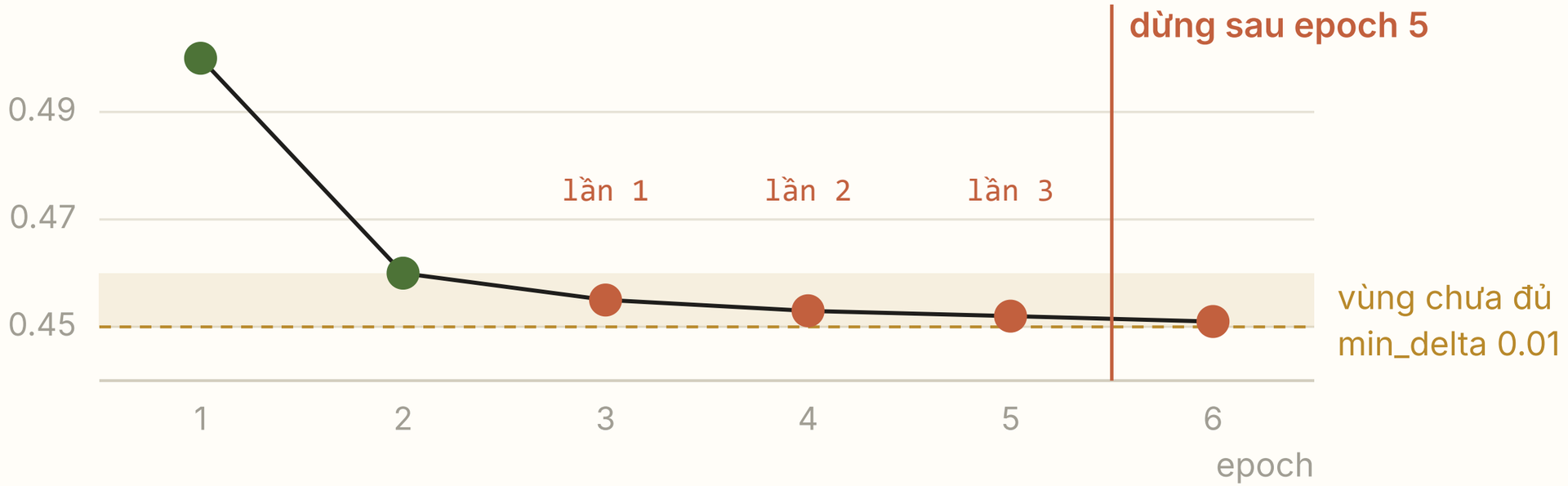
Early stopping giữ best

Patience: số lần đánh giá liên tiếp không cải thiện thì dừng.

`min_delta`: mức cải thiện tối thiểu được tính là tiến bộ.

Ví dụ: dừng ở epoch 20, tốt nhất ở epoch 12 → khôi phục **epoch 12**, không lấy model cuối.

Đếm patience



Patience 3, min_delta 0.01: tốt nhất là 0.46 ở epoch 2; ba epoch sau chỉ giảm chưa tới 0.01 (vùng tô) nên được đếm 1, 2, 3 và huấn luyện dừng sau epoch 5.

HỎI NHANH

Early stopping với patience 3 và min_delta 0.01: một lần đánh giá chỉ tính là cải thiện nếu thấp hơn loss tốt nhất ít nhất 0.01. Validation loss sau epoch 1 đến 6 là 0.50, 0.46, 0.455, 0.453, 0.452, 0.451. Huấn luyện dừng sau epoch nào?

A Sau epoch 4

B Sau epoch 5

C Sau epoch 6

D Không dừng, vì loss vẫn giảm sau mỗi epoch

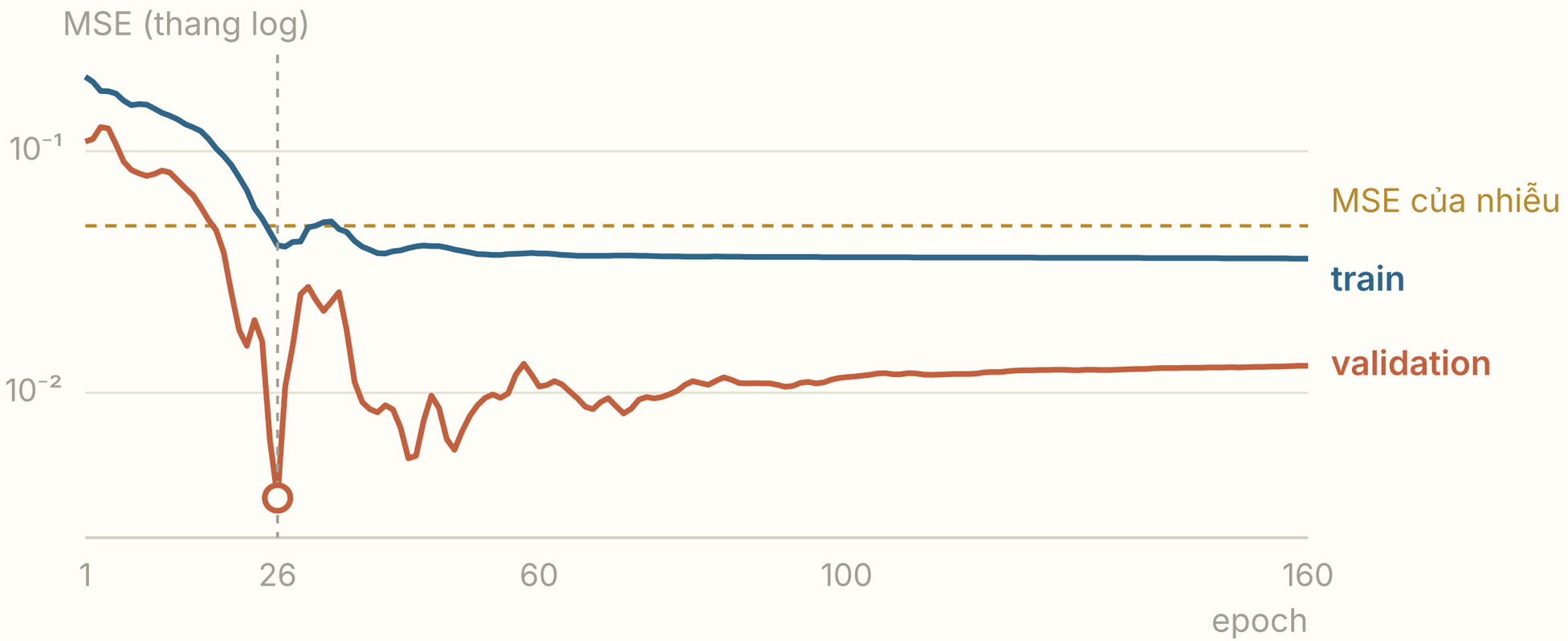
Mạng nhỏ trên 16 điểm nhiễu

Decay	Best epoch	Best MSE	Last MSE	Last train
0	26	0.00367	0.01294	0.0359
0.1	26	0.00364	0.0121	0.0361

MSE của riêng nhiễu là **0.0491**; train cuối thấp hơn mức này nên mạng đã khớp nhiễu.

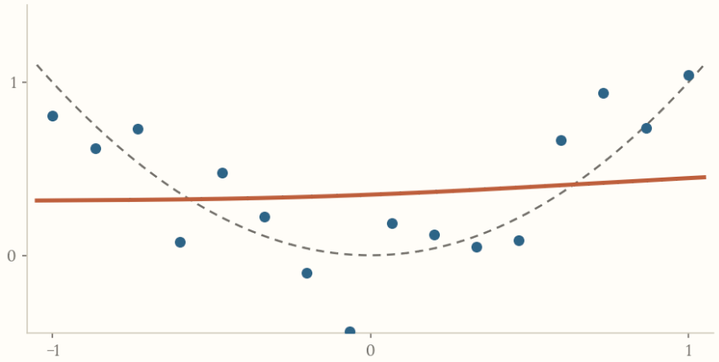
Chênh best–last lớn hơn nhiều chênh giữa hai decay.

Đường học của decay 0

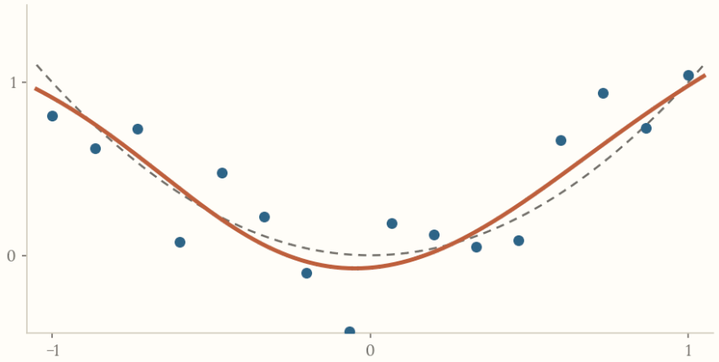


Mạng nhỏ trên 16 điểm (decay 0, số từ code bài): validation MSE chạm một đáy hẹp ở epoch 26 rồi nhích dần lên; train loss xuống dưới MSE của nhiễu, tức đã khớp một phần nhiễu.

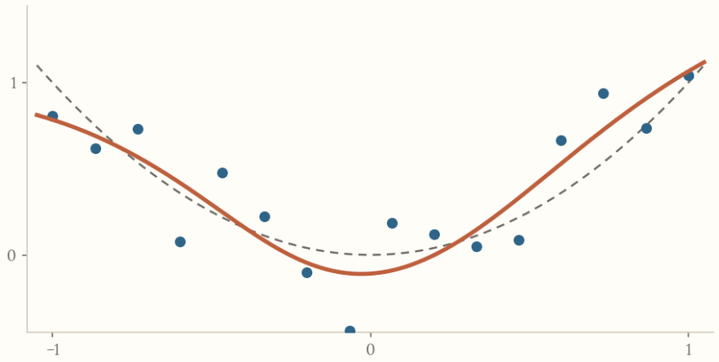
Đường cong qua từng epoch



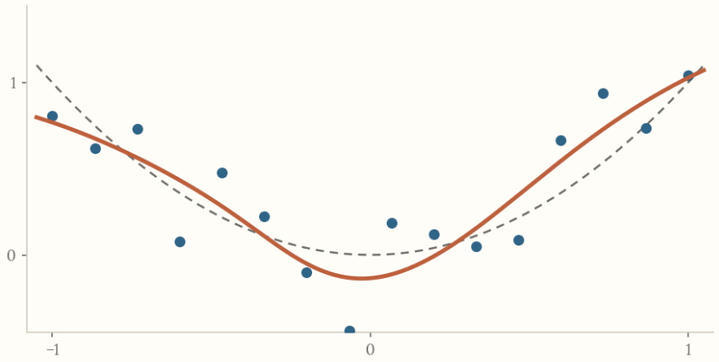
epoch 6: train MSE 0.1618, validation MSE 0.09046



epoch 26: train MSE 0.0408, validation MSE 0.00367 (tốt nhất)



epoch 80: train MSE 0.0366, validation MSE 0.01117



epoch 160: train MSE 0.0359, validation MSE 0.01294

Mạng nhỏ của bài trên 16 điểm nhiễu (chấm xanh), quy tắc thật $y = x^2$ (nét đứt): đường cam khớp nhất ở epoch 26, sau đó uốn theo nhiễu.

HỎI NHANH

Với decay bằng 0, MSE tốt nhất là 0.00367 ở epoch 26, còn MSE ở epoch cuối là 0.01294. Một bạn đề xuất lần sau train 500 epoch “cho chắc”, rồi nộp model cuối. Nên sửa đề xuất thế nào?

A Theo dõi validation, nộp checkpoint tốt nhất

B Giữ nguyên, vì train càng lâu thì train loss càng thấp

C Chỉ cần tăng decay lên 0.1 rồi nộp model cuối

D Cố định đúng 26 epoch rồi bỏ hẳn validation

03

BatchNorm

BatchNorm có buffer

Train dùng mean/variance của batch.

Eval mặc định dùng running stats.

, học thang và độ dời.

Running stats lưu trong `state_dict`, cập nhật ở train:

Momentum BN khác momentum optimizer.

Tính tay với hai số 1 và 3

Đặt `momentum=1`, , ; bỏ qua khi tính tay.

Mean · variance khi chuẩn hóa (chia) .

Running variance (không chệch, chia) .

Train cho ; eval cùng input cho $\sqrt{}$.

Kiểm bằng thư viện

Batch có hai hàng `[1,2]` và `[3,4]`: cột thứ hai có mean 3, còn hai cột đều có sample variance 2.

KẾT QUẢ

```
running mean: [2.0, 3.0]
running variance: [2.0, 2.0]
```

Train: , .

Eval: , .

Chuẩn hóa chia ; running variance chia .

Cùng input, train khác eval

batch		train: chia B		eval: running var chia B − 1	
1	2	−1	−1	−0.7071	−0.7071
3	4	1	1	0.7071	0.7071

mean (2, 3)

var: B → 1, B − 1 → 2

cùng input, khác output: running variance ghi bằng ước lượng chia B − 1

BatchNorm affine=False, momentum=1 trên hai hàng (1, 2) và (3, 4): train chuẩn hóa bằng phương sai chia B (ra ±1); running variance ghi chia B − 1 nên eval ra ±0.707.

HỎI NHANH

Ở chế độ eval, một BatchNorm mặc định, tức `track_running_stats=True`, chuẩn hóa input bằng thống kê nào?

A Mean và variance của batch eval đang chạy

B Running mean và running variance đã học trong lúc train

C Mean 0 và variance 1, cố định từ lúc khởi tạo

D Mean và variance fit lại trên toàn tập test

HỎI NHANH

`nn.BatchNorm1d` mặc định, `momentum=0.1`, vừa khởi tạo nên running mean bằng 0. Sau một lượt forward ở chế độ train với một batch có mean bằng 2, running mean bằng bao nhiêu?

A 0.2

B 1.8

C 2

D 0, vì running mean chỉ cập nhật khi eval

BatchNorm giữ thang tín hiệu

Mạng 10 lớp ReLU của bài 3.3, giữ
; std sau lớp 1, 5, 10:

KẾT QUẢ

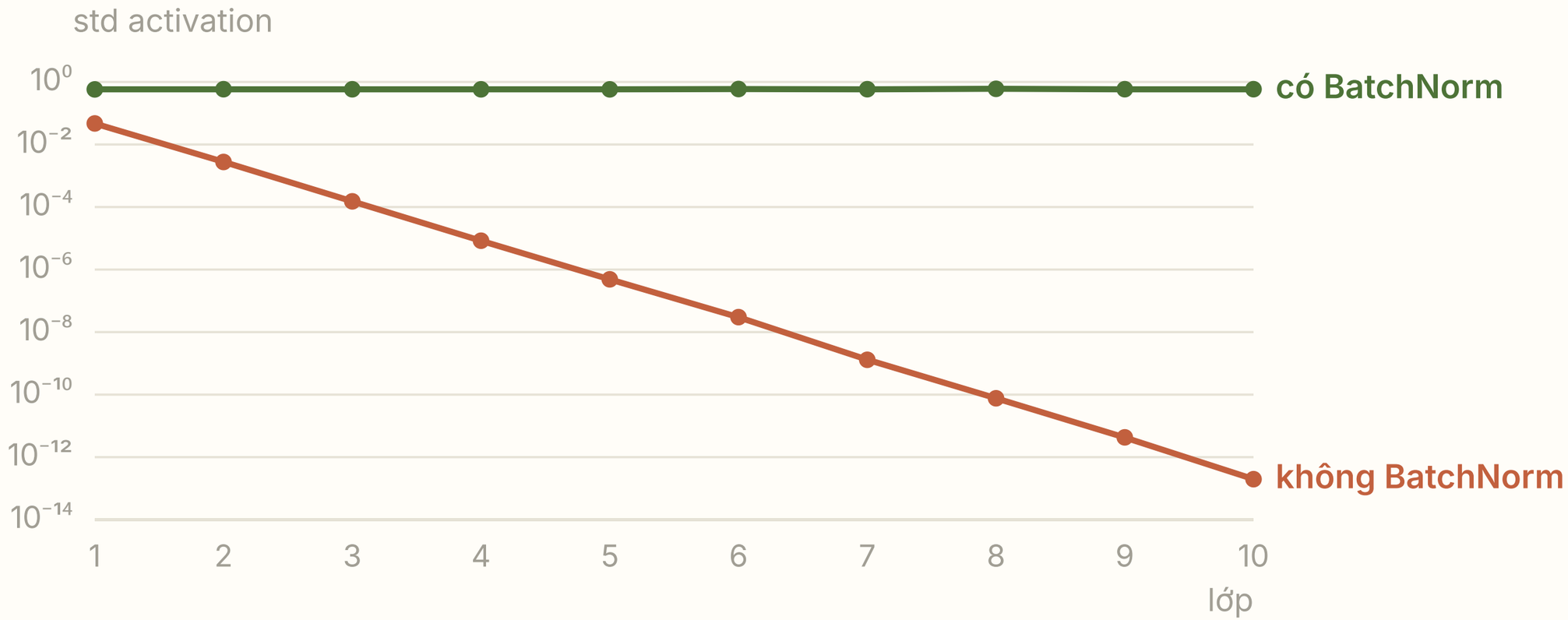
```
plain: 4.7e-02 4.9e-07 2.0e-13
with BatchNorm: 5.8e-01 5.8e-01 5.9e-01
```

Không BN: nhân khoảng mỗi lớp.

BN: mỗi lớp về mean 0, variance 1.

He sửa lúc khởi tạo; BN sửa ở mỗi lớp.

Std qua 10 lớp



10 lớp ReLU rộng 64, khởi tạo $N(0, 0.01^2)$ (số từ code bài): không BatchNorm, std co còn cỡ 10^{-13}; chèn BatchNorm sau mỗi Linear, std giữ quanh 0.58 ở mọi lớp.

Regularization chỉ là hiệu ứng phụ

Thống kê batch đổi theo mẫu đứng cùng batch.

Cùng một mẫu nhận nhiễu nhỏ khác nhau.

Dropout ngay trước BN: variance lúc train khác lúc eval.

Running variance lệch với lúc eval.

Thường đặt dropout sau phần có BN.

HỎI NHANH

Cùng một ảnh nằm trong hai batch train khác nhau cho ra hai activation sau BatchNorm khác nhau, dù trọng số chưa đổi. Nhận định nào đúng?

A Lỗi cài đặt, vì BatchNorm phải cho cùng kết quả mọi lúc

B Lỗi, vì running statistics bị ghi sai sau mỗi batch

C Bình thường, vì BatchNorm bốc mask ngẫu nhiên như dropout

D Bình thường, vì thống kê của batch thay đổi, tạo ra một chút nhiễu có ích

Batch nhỏ: chuẩn hóa trong từng mẫu

BatchNorm gộp qua batch; batch 1 mẫu trong train báo lỗi.

Batch rất nhỏ: thống kê nhiễu.

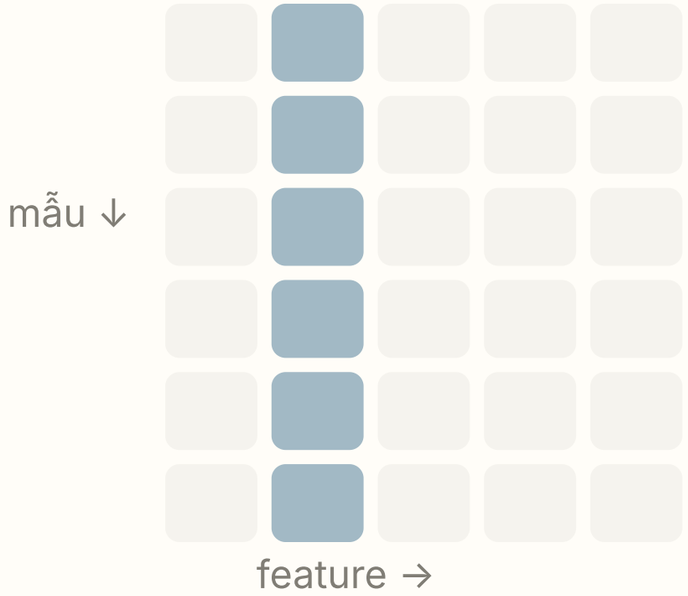
`drop_last=True` bỏ batch cuối lẻ.

LayerNorm: gộp feature của một mẫu, không running stats.

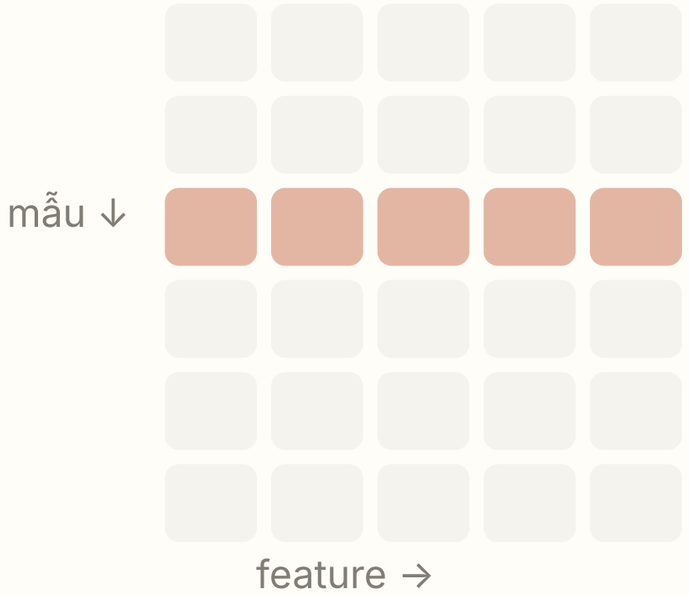
GroupNorm: từng nhóm kênh của ảnh.

Gộp theo cột hay theo hàng

BatchNorm: mỗi feature một mean



LayerNorm: mỗi mẫu một mean



Trên bảng (mẫu × feature): BatchNorm lấy mean theo cột, gộp qua các mẫu trong batch; LayerNorm lấy mean theo hàng, chỉ trong một mẫu nên không phụ thuộc batch.

HỎI NHANH

Tensor `x` có shape `(32, 10)`: 32 mẫu, 10 feature. Lúc train, `nn.BatchNorm1d(10)` và `nn.LayerNorm(10)` lần lượt tính bao nhiêu giá trị mean?

A Cả hai đều 10

B Cả hai đều 32

C BatchNorm tính 10, LayerNorm tính 32

D BatchNorm tính 32, LayerNorm tính 10

Khi đi thi

- So train/val cùng chế độ eval trước khi thêm regularization.
- Early stopping + lưu checkpoint tốt nhất; nộp từ checkpoint đó.
- `model.eval()` trước khi dự đoán; hai lần phải giống hệt.
- Dò decay, dropout trên lưới thô; đổi một yếu tố mỗi lần.
- Batch rất nhỏ: GroupNorm/LayerNorm.

04

Luyện đề

LUYỆN ĐỀ 1

Train một mạng sâu 10 lớp. Độ lớn gradient theo từng lớp, từ epoch 1 đến epoch 50, như sau. Lớp 1, gần đầu vào: 0.0003 xuống 0.0001. Lớp 5: 0.0012 xuống 0.0008. Lớp 10, ở đầu ra: 0.85 lên 1.23. Đây là vấn đề gì, và sửa cách nào hiệu quả nhất?

A Learning rate quá cao ở các lớp đầu, nên dùng learning rate riêng cho từng lớp

B Thống kê của BatchNorm bất ổn, nên tăng batch hoặc dùng GroupNorm

C Vanishing gradient ở các lớp đầu, nên thêm residual connection

D Exploding gradient ở lớp ra, nên clip độ lớn gradient về 1

ĐỀ THI NOAI 2026 Singapore · Chung kết · Câu 8

LUYỆN ĐỀ 2

`Dropout(p=0.5)` đặt ngay trước `BatchNorm1d` không có affine. Input của dropout luôn có mean 0, variance 1. Sau khi train lâu, running variance của BatchNorm xấp xỉ bao nhiêu, và lúc eval thì đầu ra của BatchNorm ra sao?

A Khoảng 1, đầu ra lúc eval có cùng thang với lúc train

B Khoảng 0.5, đầu ra lúc eval phình ra, variance khoảng 2

C Khoảng 2, đầu ra lúc eval có cùng thang với lúc train

D Khoảng 2, đầu ra lúc eval co lại, variance khoảng 0.5

PHỎNG THEO NOAI 2026 Singapore · Vòng sơ loại · Câu 195

LUYỆN ĐỀ 3

Loss của một mạng được cộng thêm số hạng – , với , và cập nhật bằng SGD với . Một trọng số có đạo hàm của phần loss dữ liệu là . Sau một bước, bằng bao nhiêu?

A 1.87

B 1.97

C 2.07

D 0.97

PHỎNG THEO CMU 10-601 · Exam 2 Practice · Q4.4(c)

Chốt bằng nơi tác động

Activation → dropout · tham số → decay.

Thời điểm → early stopping · thống kê → BatchNorm.

Train/eval và `state_dict` là một phần của model.

Sau giờ học

Làm **10 câu kiểm tra cuối bài** trên trang bài học.

Chạy notebook; đổi decay và số epoch của `fit_small`, dự đoán trước best epoch và last MSE rồi kiểm bằng output.